

Problem Statement: Constrained Object Detection & Reasoning API

Format: Individual, take-home

Submission Window: 5 days from release

Track: Computer Vision + Applied ML Engineering (with a light Agentic component)

Background

We build systems that are ~80% **GenAI/Agentic AI** and ~20% **Computer Vision / traditional ML**. This screening round exists to filter for candidates who can genuinely design, train, evaluate, and reason about an ML system end-to-end — not candidates who can produce a plausible-looking submission by chaining a tutorial and an AI coding assistant.

This round tests foundational engineering ability. It intentionally does **not** test agentic orchestration in depth — that is covered in the offline hackathon round for shortlisted candidates.

What You Are Given

- **No dataset:** You must select and justify your own.
- **No starter code, no notebook, no pretrained checkpoint.**
- **A list of hard constraints** (detailed below) that your submission will be strictly audited against.
- **Hidden Evaluation Set:** A private hold-out test set will evaluate your final API during review. You will not see these images prior to submission. This is non-negotiable and applies to all candidates equally.

The Task

Build and deploy an object detection system using **RT-DETR**, on a domain and dataset of your choosing, exposed through a working API — plus a very small reasoning layer on top of it.

Part A — Detection

1. **Choose a real-world object detection problem** in a domain that interests you (traffic, retail shelves, wildlife, sports, agriculture, safety equipment, etc.). It must involve **at least one class that is not a standard COCO class**, so you cannot simply run an off-the-shelf pretrained model and call it done.
2. **Source and prepare your own dataset** (public dataset, scraped and self-labeled, or a mix). Document exactly where your data came from and how you split it.
3. **Fine-tune RT-DETR** on your dataset.
4. **Evaluate using appropriate metrics** (mAP, precision/recall, confusion behavior on your classes) — self-reported metrics carry less weight than performance on the hidden set (see Evaluation).

5. **Save your trained model and expose it through a FastAPI endpoint** that accepts an image and returns detected objects, bounding boxes, and confidence scores.

Part B — Minimal Reasoning Layer (no frameworks)

Add a second endpoint that accepts a **natural-language question** about an image (e.g., *"How many people are in this image?"*, *"Is anyone not wearing a helmet?"*, *"What's the most common object here?"*) and executes the following logic:

- **Intent Routing:** Decides whether the question requires calling your detection model, or can be answered without it (e.g., a question unrelated to the image content).
- **Structured Reasoning:** If detection is needed, calls your Part A model, then reasons over the structured output (boxes/classes/counts) to answer in plain language.
- **Confidence Guardrail:** If the detection results are insufficient to answer confidently, the response must say so explicitly rather than guessing.

Note: This is a **single hand-written decision layer** — not a multi-agent system. It checks that you understand how to connect a model's structured output to an LLM-based reasoning step yourself, without a framework doing it for you.

Hard Constraints

1. **No agentic frameworks:** LangChain, LangGraph, CrewAI, AutoGen, or similar are strictly prohibited anywhere in this submission, including Part B.
2. **No AutoML / no-code training platforms:** Training must be your own code, using RT-DETR (any faithful implementation/library is fine — e.g., Ultralytics RT-DETR, Hugging Face Transformers, PaddleDetection).
3. **At least one non-COCO class:** Submissions detecting only standard COCO classes with an unmodified pretrained model will score zero on Part A.
4. **Reproducibility is mandatory:** You must include exact steps, environment/requirements, hardware used, training time, and hyperparameters. If we cannot reproduce your training run from your instructions, Part A is capped at 50%.
5. **Tools & Assistance:** You may use AI coding assistants (ChatGPT, Copilot, Claude, etc.) and the internet freely. There is no restriction on tools — only on your ability to explain and justify what they produced (see Deliverables and Review).

Deliverables

Submit all of the following through the submission form:

- **1. Source code (GitHub repo):** Including training script, evaluation script, and FastAPI app for both endpoints.
- **2. Model weights:** Or a direct, working download/load path (do not make reviewers guess).
- **3. Written memo (max 2 pages) covering:**
 - Why you chose this domain/dataset, and how you sourced/labeled it.
 - Your train/val/test split strategy and technical justification.
 - Your evaluation metrics and what they actually tell you (and what they don't).

- **Five failure cases** from your own model, with detailed root-cause analysis (blur, small object, occlusion, class confusion, lighting, etc.). This is strictly graded — a model with zero acknowledged failure cases is treated as a red flag, not a strength.
 - How your Part B reasoning layer decides when to call the detector vs. not, and one specific example where it correctly outputs "insufficient information."
- **4. API usage instructions:** Clear run instructions, complete with sample request/response payloads for both endpoints.

Evaluation Criteria

Component	Weight
Performance on the hidden evaluation set (Part A)	25%
Dataset sourcing, labeling quality, and justification	15%
Evaluation methodology and honesty of self-reported metrics	10%
Quality of failure-case analysis in the memo	15%
Part B reasoning layer — correctness and honest "insufficient info" handling	15%
Code quality, reproducibility, API implementation	10%
Bonus: Docker containerization, VM/server deployment, logging, error handling	10%

Note on grading philosophy: We are not primarily grading mAP on your own test set. Two submissions with similar self-reported metrics can score very differently once run against the hidden set and once we read how well you understood your own model's limits. A candidate who reports 78% mAP with three well-reasoned failure cases will be ranked above one who reports 95% mAP with no acknowledged weaknesses.

Notes & Policies

- **Individual submission:** Submissions with substantially identical code, dataset choice, and memo content across candidates will be flagged for both parties.
- **Verbal Defense:** Shortlisted candidates should expect to explain and defend any part of this submission verbally in the offline round — including specific lines of code, why a class was chosen, and how the reasoning layer would behave if given an ambiguous question.
- **Mid-project pivots:** If your dataset choice or approach changes mid-way, briefly note in the memo what you tried first and why you changed course — this is useful signal, not a penalty.